T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	02/10/2026	**DATE SUBMITTED:**	03/10/2026
NAME:	Uchi, Mao Sophie S. Verano, James Daniel M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.3. Adjacency List

Code

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

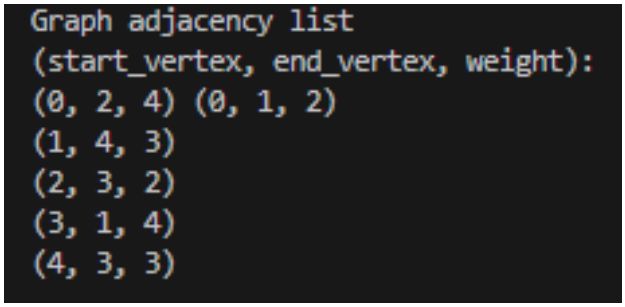
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    //insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        // point new node to current head
        newNode->next = head;
        return newNode;
    }
    // number of nodes in the graph
    int N;
    public:
    //adjacency list as array pof pointers
    adjNode **head;
    //constructor
```

```
30      DiaGraph(graphEdge edges[], int n, int N) {
31          // allocate new node
32          head = new adjNode*[N] ();
33          this->N = N;
34          //initialize head pointer for all vertices
35          for (int i = 0; i < n; i++) {
36              head[i] = nullptr;
37          }
38          //construct directed graph by adding edges to it
39          for(unsigned i = 0; i < n; i++) {
40              int start_ver = edges[i].start_ver;
41              int end_ver = edges[i].end_ver;
42              int weight = edges[i].weight;
43              //insert in the beginning
44              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
45              // point head pointer to new node
46              head[start_ver] = newNode;
47          }
48      }
49      // Destructor
50      ~DiaGraph() {
51      for (int i = 0; i < N; i++)
52          delete[] head[i];
53          delete[] head;
54      }
55  };
56
57  void display_AdjList(adjNode* ptr, int i){
58      while(ptr != nullptr) {
59          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
60          ptr = ptr->next;
61      }
62      std::cout << std::endl;
63  }
```

```
64  //graph implementation
65  int main() {
66
67      // graph edges array.
68      graphEdge edges[] = {
69      // (x, y, w) -> edge from x to y with weight w
70      {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
71      };
72      // Number of vertices in the graph
73      int N = 6;
74      // calculate number of edges
75      int n = sizeof(edges)/sizeof(edges[0]);
76      // construct graph
77      DiaGraph diagraph(edges, n, N);
78      // print adjacency list representation of graph
79      std::cout<<"Graph adjacency list "<< std::endl << "(start_vertex, end_vertex, weight):" << std::endl;
80      for (int i = 0; i < N; i++) {
81      // display adjacevint vertices of vertex i
82      display_AdjList(diagraph.head[i], i);
83      }
84      return 0;
85  }
```

Output	

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Code (Header)

```cpp
#ifndef DEPTH_H
#define DEPTH_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
return os;
}

```

```cpp
template <typename T>
class Graph {
    public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }

    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
    // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
        edge_list.emplace_back(e);
        } else {
        std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
            edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
```

```cpp
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
        private:
        // Stores number of vertices in graph
        size_t V;
        std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest){
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    //Assume that DFS always starts from vertex ID 1
    stack.push(1);
    while(!stack.empty()){
        auto current_vertex = stack.top();
        stack.pop();
        //If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e: G.outgoing_edges(current_vertex)){
                //If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()){
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
```

```
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
        for (auto &i : edges) {
            for (auto &j : i.second) {
                G.add_edge(Edge<T>{i.first, j.first, j.second});
            }
        }
    return G;
}

template <typename T>
void test_DFS() {
    //Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " <<std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
    std::cout << v << std::endl;
}

#endif
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

Code (Header)

```
#ifndef BREADTH_H
#define BREADTH_H
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    } inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {

    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph {
```

```
41  public:
42      Graph(size_t N) : V(N) {}
43      auto vertices() const {
44          return V;
45      } auto &edges() const {
46          return edge_list;
47      } void add_edge(Edge<T> &&e) {
48          if (e.src >= 1 && e.src <= V &&
49              e.dest >= 1 && e.dest <= V)
50              edge_list.emplace_back(e);
51          else
52              std::cerr << "Vertex out of bounds" << std::endl;
53      }
54
55      auto outgoing_edges(size_t v) const {
56          std::vector<Edge<T>> edges_from_v;
57          for (auto &e : edge_list) {
58              if (e.src == v) {
59                  edges_from_v.emplace_back(e);
60              }
61          }
62          return edges_from_v;
63      }
64
65      template <typename U>
66      friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);
67
68  private:
69      size_t V;
70      std::vector<Edge<T>> edge_list;
71  };
72
73  template <typename T>
74  auto create_reference_graph() {
75      Graph<T> G(9);
76
77      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
78      edges[1] = {{2, 2}, {5, 3}};
79      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
80      edges[3] = {{4, 2}, {7, 3}};
```

```
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

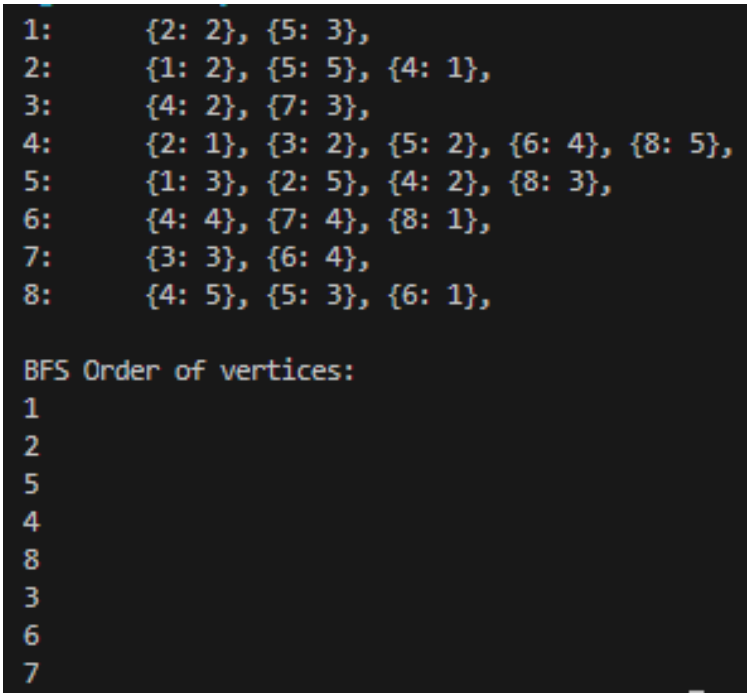
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            queue.push(e.dest);
            }
        }
        return visit_order;
    }

```

```
template <typename T>
void test_BFS() {

    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
      std::cout << v << std::endl;
}

#endif
```

Output	

Procedural Activity Overall Analysis

In accomplishing the code implementation of each ILO's, Each searching algorithm has their own unique capabilities. They all follow different strategy rules, applying data structure into their code implementation. Starting from:

ILOA: Graph Implementation utilizing Adjacency List

The sample code that was applied and represented focuses on constructing a directed, weighted graph using an adjacency list representation. It has code structures that serve as the starting foundation such as adjNode to dynamically link a custom node to form an array of pointer heads. There is a familiar structure that is inherited through the learning of binary trees and nodes, which is the creation and insertion logic. It is indicated in certain functions such as the function **ajNode* getAdjListNode()** Which contains the insertion of a new node indicated **newNode = new adjNode**, And the allocation of the head for when the insertion happens which is indicated in the function **DiaGraph()**, containing the **head = adjNode*[N]()**. In relating this function to our current topic, introducing the basics once again will be a building block for the adjacency list as for its inserting logic on new edges. It is represented as **newNode->next = head[start_ver]**, which starts at the beginning (head) of the linked list, and the printed order for each vertex will appear inverted compared to their declaration order in the edges array.

ILOB1: Depth-First Search (DFS) Traversal

Now in the Depth-First Search (DFS) algorithm. This algorithm implements an iterative, **stack-based traversal** starting automatically from **vertex 1**. It is indicated in the code as **stack.push(1)**. The rules that this algorithm follows is the data structure restriction on the stack list. Therefore, it follows the **Last-in, First Out (LIFO) order**. The node pushed last becomes the top of the stack and is processed first during the exploration of nodes / vertices leading down to the deep branch of the graph before backtracking. The program loops through the outgoing edges, vertices are discovered and are pushed onto the stack sequentially according to whether they have been visited or not. This algorithm is best applied for ideal maze routes and identifying connectivity components just how an example in the supplementary is applied through DFS.

ILOB2: Breadth-First Search (BFS) Traversal

Lastly is the Breadth-First Search (BFS) algorithm. This algorithm shifts the tracking to an iterative, **queue-based traversal strategy**. Compared the ILOB which uses a **stack-based traversal strategy**. Compared to the DFS, The BFS does not have the capabilities to backtrack as it follows the strict rules of a queue order list. Which was the **First-In, First Out (FIFO) order**. Same with the DFS, it starts for **vertex 1**, and in enqueueing nodes, neighbors are enqueued layer-by-layer due to the strict order of the queue list. The graph is searched across its local neighborhood layer before advancing deeper into the other layers of neighborhoods. It first traverses all the neighboring nodes inside that specific neighbor the current ptr is at, and once it has traversed all the nodes on that neighborhood, it will proceed on the next neighborhood in the below level. This BFS algorithm strategy might not be able to backtrack, but it can guarantee to find the shortest path on unweighted structural layouts.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

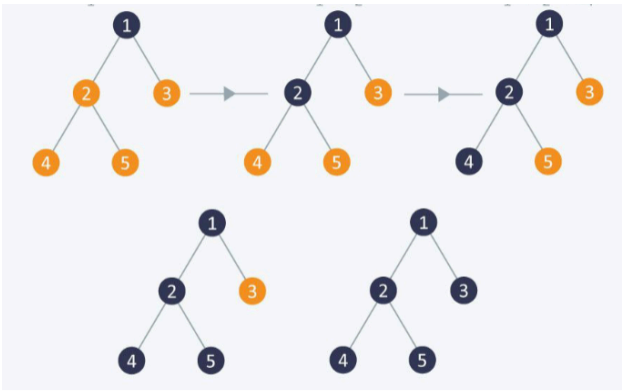
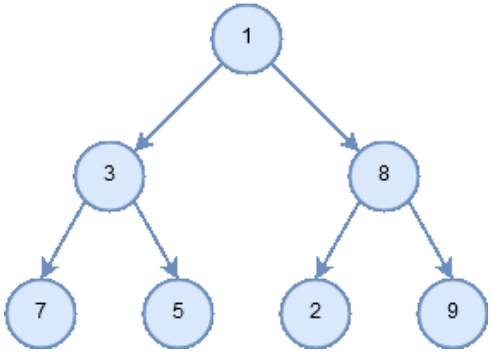
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The algorithm that best fits in helping to accomplish the task is the Depth-First Searching algorithm (DFS). As tackled, the DFS algorithm is a **recursive algorithm that uses the idea of backtracking**. Which perfectly fits the scenario of the person wanting to visit different locations indicated on a map. Starting from one location (vertex) and wanting to visit every vertex and backtracking the vertex locations again, then explore other vertices from the same vertex. With the DFS algorithm representing hitting a dead end or a path that currently does not have any more nodes to traverse on, the DFS algorithm makes it possible to backtrack the same node you've come from.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of Depth-First Search(DFS) in traversal strategies for trees are called preorder, inorder, and postorder strategies. Unlike BFS, DFS and Traversal Strategies dig down from a singular path until all its lower neighbors(leaf nodes) are traversed; backtracking to visit the remaining unvisited node branches from a higher level to complete its search. The data structure applied for DFS is Stack, as the search follows the rule LIFO or Last In, First Out; same goes for Traversal Strategies. The difference between DFS and Traversal Strategies, Traversal Strategies' objective is to search for each node in an orderly manner, while DFS does not follow this goal, but to explore every lower node from a single path until it is not possible to traverse more.

A Graphical Comparison between DFS and Traversal Strategies

Depth-First Search	Traversal Strategies
	

Traversal Strategies Pseudocode

```
START
FUNCTION preOrder
  SET left_Node, right_Node, key
  DISPLAY key
  IF left_Node does not equal to nullptr
    left_Node points to preOrder
  IF right_Node does not equal to nullptr
    right_Node points to preOrder
END

START
FUNCTION inOrder
  SET left_Node, right_Node, key
  IF left_Node does not equal to nullptr
```

	left_Node points to inOrder DISPLAY key IF right_Node does not equal to nullptr right_Node points to inOrder END START FUNCTION **postOrder** SET left_Node, right_Node, key IF left_Node does not equal to nullptr left_Node points to postOrder IF right_Node does not equal to nullptr right_Node points to postOrder DISPLAY KEY END

Traversal Strategies Code Implementation

Pre-Order

```
50  template <typename T>
51  void BinarySearchTree<T>::preOrder(){
52      std::cout << key << " ";
53      if(left != nullptr){
54          left->preOrder();
55      }
56      if(right != nullptr){
57          right->preOrder();
58      }
59  }
```

In-Order

```
61  template <typename T>
62  void BinarySearchTree<T>::inOrder(){
63
64      //go to left
65      if(left != nullptr){
66          left->inOrder();
67      }
68      std::cout<<key<< " ";
69      //go to right
70      if(right != nullptr){
71          right->inOrder();
72      }
73  }
```

Post-Order

```
75  template <typename T>
76  void BinarySearchTree<T>::postOrder(){
77
78      //go to left
79      if(left != nullptr){
80          left->postOrder();
81      }
82      //go to right
83      if(right != nullptr){
84          right->postOrder();
85      }
86      std::cout<<key<< " ";
87  }
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

The data structure used to implement the Breadth First Search(BFS) is Queue. In order to be familiar with how BFS operates, with Queue data structure, the search will follow the rule FIFO or First In, First Out. BFS algorithm relies on exploring the graph level-by-level while similarly to Queue, the first element in a queue must be the first one to be enqueued(added) and dequeued(removed). To find the shortest path in a graph, FIFO is used to process elements level-by-level, basically the nearest vertex from the current vertex will be visited next. Both queues grow and shrink whenever a new element has been visited/appended until the queue is empty. A directed graph has an order of pattern that prioritizes a level before moving on to the next; utilizing FIFO will prevent leaving out a vertex before exploring every path.

4. How many times can a node be visited in the BFS?

A Breadth-First Search algorithm can only visit nodes **ONCE**. As the definition states in the tackled presentation in the module, BFS is a traversing algorithm where you should start traversing from a selected node and traverse the graph **LAYERWISE**. Thus, representing that before proceeding to another next-level neighbour nodes, you first have to explore the neighbouring nodes first on the current level you are in. Which means that, you can only visit nodes once, since compared to the Depth-First Search algorithm (DFS), the BFS does not have the ability to backtrack nodes.

Uchi's Conclusion

My conclusion for this activity is knowing the two common Graph Traversal Algorithms, how to implement both Breadth First Search(BFS) and Depth-First Search(DFS) and its importance. Breadth First Search explores nodes and its neighbors layer-by-layer, meaning this search visits a node close from the current node; Its data structure implementation uses Queue, with a rule FIFO or First In, First Out. Depth-First Search, from the name itself, deep dives into all the lower neighbors of the current node before getting back to the unvisited nodes; Its data structure implementation uses Stack, with a rule LIFO or Last In, First Out. For a proper visitation of each vertex, both searches must start at a node that doesn't have at least 2 or more ray/lines that are connected to a node from another node. With the previous statement, a certain data structure will assist you on visiting the entirety of a graph's vertices. Both algorithms utilize a helper data structure called (Extra) Visited Array, monitoring the graph's cycle using graph traversal algorithms; a node could possibly be revisited again and again, which will cause errors from your search. This data structure is reliable at preventing never-ending loops, processing/visiting the node once.

Verano's Conclusion

In accomplishing this activity, more specifically the analysis of the code structure, I was able to deepen my knowledge how the traversal of the **Depth-First Searching (DFS) algorithm** and the **Breadth-First Searching (BFS) algorithm** works, and what are their own unique capabilities in traversing the represented graphs. Starting from the DFS, which is uniquely applied to a **stack-based** data structure and has a structural objective of traversing deeply along a unified lineage path until hitting a dead end before backtracking. Compared to the BFS, which follows a **Queue-based** data structure and has a structural objective of exploring the layout layer-by-layer, check all close-proximity neighbors first before proceeding into a deeper level of layer. It does not have the capability to backtrack, but it guarantees to find the shortest path on a specific unweighted structural layouts. In terms of Graphs representations, I learned more about selecting the right data structure depending entirely on the graph. Based on my research, an adjacency list is highly space-efficient for sparse networks because it scales dynamically using linked list nodes. An adjacency matrix offers a time complexity of **O(1)** edge lookups but introduces an unavoidable **O(V^2)** spatial penalty making it wasteful for graphs with fewer connections (GeeksforGeeks, 2025).

REFERENCES

GeeksforGeeks. (2025, July 15). *Comparison between Adjacency List and Adjacency Matrix representation of Graph*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/comparison-between-adjacency-list-and-adjacency-matrix-representation-of-graph/

GeeksforGeeks. (2025b, October 29). *Representation of graph*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/graph-and-its-representations/

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Specific grammar improvisation to represent how the code flows.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.